

PRESENTACIÓN

Modelos de clasificación: Regresión Logística, Gradient Boosting y Naive Bayes

Intuición, matemática y aplicación

Profesor Ing. Pablo F. Sciolla

Universidad de San Andrés — Analítica de Datos

Mayo 2026

0
1

REPASO Y AGENDA

De Árboles, KNN y RF a tres familias nuevas

02

De lo visto, a lo que viene

Tres familias nuevas para completar nuestro repertorio de clasificadores: un modelo lineal probabilístico, una técnica de ensemble por boosting y un clasificador generativo.

YA VISTO

Lo que ya manejan

Árboles de decisión Particiones recursivas del espacio según ganancia de información.

KNN Clasificación por similitud a los k vecinos más cercanos.

Random Forest Ensemble por bagging de árboles con submuestreo aleatorio.

AGENDA DE HOY

Lo que vamos a sumar

Bagging Ensemble que hace bootstrapping invocando a otros modelos.

Regresión Logística Modelo lineal que estima probabilidades de pertenencia a una clase.

Naive Bayes Clasificador probabilístico basado en el teorema de Bayes.

Gradient Boosting Ensemble secuencial que corrige errores residuales del modelo previo.

ENSEMBLE EN PARALELO

Bagging

03

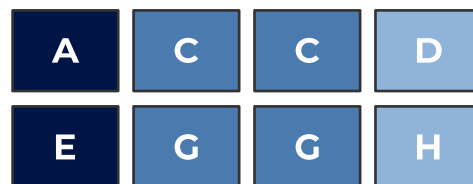
Bootstrapping: muestrear con reemplazo

Es la técnica de remuestreo que permite que cada árbol de un Random Forest vea un dataset levemente distinto. La misma idea está detrás de la estimación de intervalos de confianza por bootstrap.

Dataset original

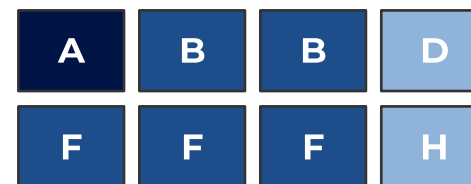


Bootstrap 1



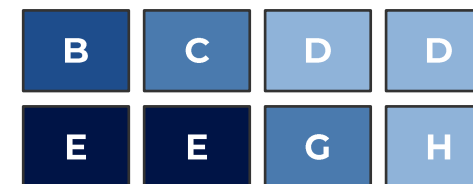
Duplicados: C×2 G×2
Sin elegir: B, F

Bootstrap 2



Duplicados: B×2 F×3
Sin elegir: C, E, G

Bootstrap 3



Duplicados: D×2 E×2
Sin elegir: A, F

Reemplazo Una misma observación puede aparecer varias veces o no aparecer.

≈ 63% únicas En cada muestra bootstrap, en promedio el 63,2% de las observaciones son únicas.

Out-of-bag El 36,8% restante (OOB) funciona como validación gratuita en cada árbol.

POR QUÉ NOS IMPORTA Random Forest entrena un árbol por cada muestra bootstrap; la diversidad entre muestras es la que hace que el promedio reduzca varianza.

05

ALGORITMO · INTUICIÓN

Bagging

Bootstrap AGGregatING: entrenar muchos modelos en paralelo sobre muestras bootstrap y promediar sus predicciones.

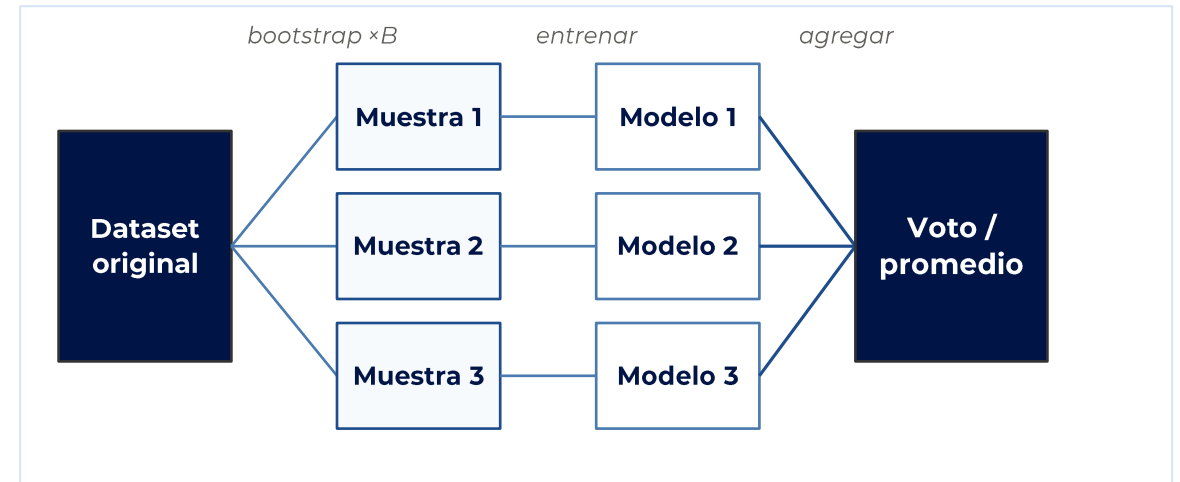
La idea

Un único árbol profundo aprende demasiado del dataset específico: tiene varianza alta. Pequeñas perturbaciones en los datos lo cambian mucho. Bagging aprovecha esa fragilidad: entrena B árboles, cada uno sobre una muestra bootstrap distinta, y promedia (o vota) sus predicciones. La varianza del promedio es menor que la de cada árbol individual — siempre que los árboles sean razonablemente diversos entre sí.

Supuestos clave

- El modelo base tiene varianza alta (árboles profundos, KNN con k chico)
- Las muestras bootstrap generan modelos suficientemente diversos
- B suficientemente grande (típicamente 100–500)

Esquema



Fortalezas

- Reduce varianza sin aumentar sesgo
- Entrenamiento paralelizable
- OOB sirve como validación gratuita

Limitaciones

- No ayuda con modelos de bajo sesgo y baja varianza
- Los árboles correlacionan (RF lo corrige)
- Pérdida de interpretabilidad

Bagging vs Random Forest

Random Forest es bagging aplicado a árboles con un agregado: en cada split, se sortea un subconjunto de features candidatas.

BAGGING
genérico, cualquier modelo base

En cada split, el árbol considera:

f₁

f₂

f₃

f₄

f₅

Las 5 features son evaluadas; se elige la mejor para el split.

Consecuencia: si una feature es muy predictiva, todos los árboles la eligen en el split raíz y quedan parecidos.

RANDOM FOREST
bagging + muestreo de features en cada split

En cada split, el árbol considera:

f₁

f₂

f₃

f₄

f₅

Solo m features sorteadas (acá m = 2 de 5); se elige la mejor entre ellas.

Consecuencia: los árboles desarrollan estructuras distintas. Menor correlación → mejor reducción de varianza al promediar.

	BAGGING	RANDOM FOREST
Modelo base	Cualquiera (árboles, KNN, LR...)	Solo árboles de decisión
Muestreo de filas	Sí (bootstrap)	Sí (bootstrap)
Muestreo de columnas (features)	No: cada split considera todas	Sí: en cada split, m de p features
Correlación entre modelos	Alta si hay una feature dominante	Más baja (más diversidad)

Bagging — formulación

La matemática

$$f_{\beta}(x) = \text{entrenar}(D_{\beta}^*) \quad \text{con } b = 1, \dots, B$$

Cada modelo f_{β} se entrena con una muestra bootstrap D_{β}^* del dataset original.

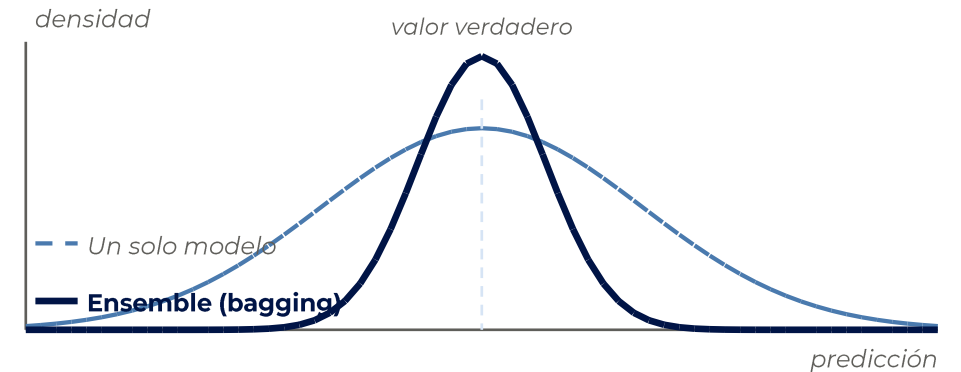
$$\hat{f}(x) = (1/B) \cdot \sum_{\beta} f_{\beta}(x) \quad (\text{regresión})$$

Para regresión, el ensemble es el promedio de las predicciones individuales.

$$\hat{c}(x) = \text{mode}\{f_{\beta}(x) : b = 1, \dots, B\} \quad (\text{clasif.})$$

Para clasificación, voto mayoritario: la clase predicha más veces gana.

Lectura visual



$$\text{Var}[\hat{f}] = \rho \cdot \sigma^2 + (1 - \rho) \cdot \sigma^2 / B$$

Por qué reduce varianza

Si los B modelos fueran independientes y cada uno tuviera varianza σ^2 , el promedio tendría varianza σ^2/B . En la práctica los modelos están correlacionados ($\rho > 0$): la varianza del ensemble es $\rho \cdot \sigma^2 + (1-\rho) \cdot \sigma^2/B$. Por eso reducir ρ — la correlación entre modelos — importa tanto como agregar más modelos.

Bagging paso a paso

Tres árboles entrenados sobre tres muestras bootstrap distintas predicen la clase de una observación nueva. El ensemble decide por voto mayoritario.

Árbol 1

entrenado con Muestra 1

Reglas aprendidas:

Si $\text{feature}_1 > 0,5$
y $\text{feature}_2 \leq 2,3$
→ Clase A

Predice:

Clase A

Árbol 2

entrenado con Muestra 2

Reglas aprendidas:

Si $\text{feature}_1 > 0,5$
y $\text{feature}_3 = \text{'sí'}$
→ Clase A

Predice:

Clase A

Árbol 3

entrenado con Muestra 3

Reglas aprendidas:

Si $\text{feature}_2 \leq 2,3$
y $\text{feature}_4 < 100$
→ Clase B

Predice:

Clase B

Agregación: voto mayoritario

A

Árbol 1

A

Árbol 2

B

Árbol 3

Conteo: A → 2 votos · B → 1 voto

Clase mayoritaria: A

Predicción del ensemble

Clase A

$$\hat{c}(x) = \text{mode} \{ A, A, B \} = A$$

Bagging — en scikit-learn

BaggingClassifier de *sklearn*: ensemble genérico que permite bagging sobre cualquier estimador base. Aquí usamos árboles de decisión profundos como modelo base.

```
python

from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

# Modelo base: un árbol profundo (alta varianza, perfecto para bagging)
arbol_base = DecisionTreeClassifier(max_depth=None)

# Bagging: 100 árboles entrenados en paralelo sobre muestras bootstrap
modelo = BaggingClassifier(
    estimator=arbol_base,
    n_estimators=100,
    max_samples=1.0, bootstrap=True,
    oob_score=True, n_jobs=-1, random_state=42,
)

modelo.fit(X_train, y_train)

# Validación out-of-bag (gratis, sin tener que separar un set)
print(f'OOB score: {modelo.oob_score_:.3f}')
```

Hiperparámetros clave

estimator

Modelo base a baggear. Cualquier estimator de *sklearn*; árboles profundos son la elección típica.

n_estimators

Cantidad *B* de modelos en el ensemble. Más es mejor, hasta un plateau (típico 100–500).

max_samples

Tamaño de cada muestra bootstrap (1.0 = mismo tamaño que el dataset original).

oob_score

Si True, calcula el error con las observaciones out-of-bag de cada árbol. Validación gratis.

MODELO LINEAL PROBABILÍSTICO

Regresión Logística

04

Regresión Logística

Modelo lineal en los log-odds que estima la probabilidad de pertenecer a una clase.

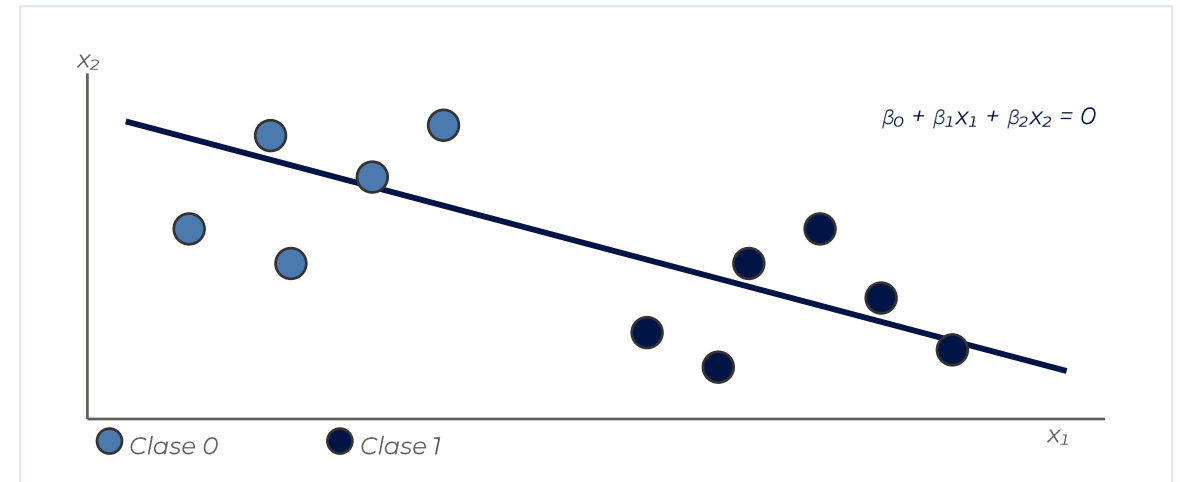
La idea

Aunque su nombre dice 'regresión', es un clasificador. La idea: combinar las features de forma lineal para obtener un puntaje z , y pasarlo por la función sigmoide para llevarlo al intervalo $(0, 1)$. Ese valor se interpreta como la probabilidad de pertenecer a la clase positiva. La frontera de decisión es un hiperplano: separa el espacio en dos regiones según el lado del cero del puntaje z .

Supuestos clave

- Linealidad entre features y log-odds
- Observaciones independientes
- Baja multicolinealidad entre predictores

Esquema



Fortalezas

- Coeficientes interpretables como odds-ratios
- Entrega probabilidades calibradas
- Rápido y robusto en alta dimensión

Limitaciones

- Falla con relaciones no lineales
- Sensible a outliers extremos
- Necesita escalar features

Regresión Logística — formulación

La matemática

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Combinación lineal de las features (los β son los coeficientes a aprender).

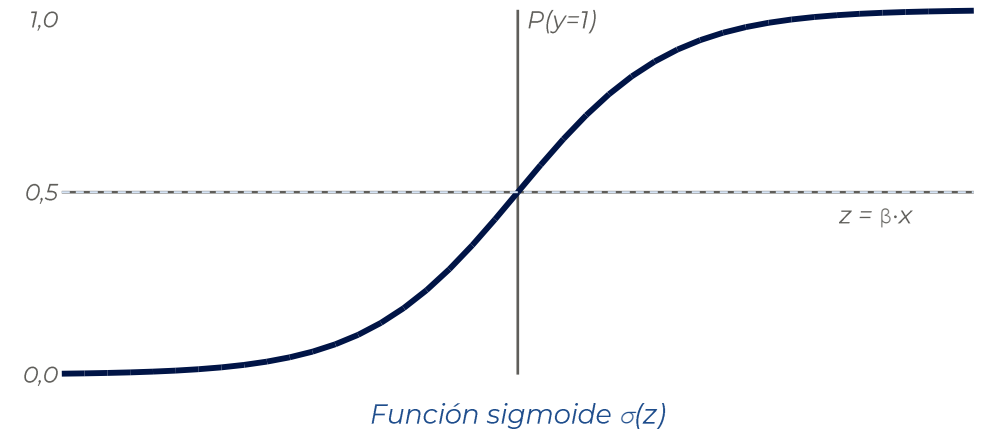
$$P(y=1 | x) = \sigma(z) = 1 / (1 + e^{-z})$$

La sigmoide aplasta z al intervalo $(0, 1)$ para interpretarlo como probabilidad.

$$L(\beta) = - \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Pérdida de log-verosimilitud (cross-entropy): la que se minimiza al ajustar.

Lectura visual



Cómo se entrena

Los coeficientes β se estiman maximizando la verosimilitud (equivalente a minimizar la cross-entropy) mediante optimización numérica iterativa, típicamente Newton-Raphson o gradiente. No hay solución cerrada como en regresión lineal.

Regresión Logística — caso de uso

Predecir la probabilidad de incumplimiento (default) de un crédito personal a 12 meses.

El contexto

Un banco minorista necesita un score automático para aprobar o rechazar solicitudes de préstamo, y debe poder explicar el motivo a regulador y cliente.

Los datos

≈ 200.000 solicitudes históricas con variables sociodemográficas, ingresos, historial crediticio y comportamiento de pago. Etiqueta binaria: default a 12 meses.

Por qué este modelo

Necesitamos probabilidades calibradas para fijar umbrales por segmento, e interpretabilidad para el comité de riesgo y el regulador. La logística entrega ambas.

Cómo se evalúa

AUC y KS para discriminación; calibración (curva de confiabilidad); además, lift por decil del score para fijar el punto de corte de aprobación.

A TENER EN CUENTA *Si las variables no son aproximadamente lineales en log-odds, conviene transformar (binning WoE, splines) o pasar a un modelo no lineal como Gradient Boosting con SHAP para preservar interpretabilidad.*

Regresión Logística — en scikit-learn

Implementación mínima en sklearn: pipeline con escalado, ajuste, predicción de clase y de probabilidad.

```
python

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score

# Pipeline: escalado + modelo
modelo = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(penalty='l2', C=1.0)),
])

# Ajuste
modelo.fit(X_train, y_train)

# Predicción de clase y de probabilidad
y_pred = modelo.predict(X_test)
y_proba = modelo.predict_proba(X_test)[ :, 1]

auc = roc_auc_score(y_test, y_proba)
```

Hiperparámetros clave

penalty

Tipo de regularización: 'l2' (default), 'l1', 'elasticnet' o 'none'.

C

Inverso de la fuerza de regularización: C bajo → regularización fuerte.

solver

'lbfgs' por defecto; 'liblinear' para datasets chicos; 'saga' soporta L1 a escala.

class_weight

'balanced' compensa clases desbalanceadas automáticamente.

CLASIFICADOR PROBABILÍSTICO

Naive Bayes

05

04

ALGORITMO · INTUICIÓN

Naïve Bayes

Clasificador generativo basado en Bayes con un supuesto fuerte: independencia condicional entre features dada la clase.

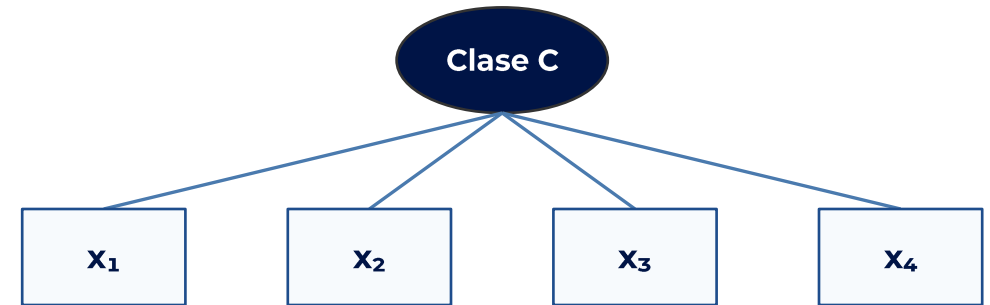
La idea

Para clasificar una nueva observación, calcula la probabilidad posterior de cada clase usando el teorema de Bayes. El supuesto 'naïve' es asumir que las features son independientes entre sí una vez fijada la clase. Es una simplificación dura — y muchas veces falsa — pero permite descomponer la verosimilitud conjunta como un producto, y eso lo hace extremadamente rápido, escalable y sorprendentemente competitivo.

Supuestos clave

- Independencia condicional dada la clase
- Distribución asumida por feature: Gaussiana, multinomial o Bernoulli
- Suaviza con Laplace para evitar ceros

Esquema



Supuesto naïve: dada la clase, las features son condicionalmente independientes

Fortalezas

- Entrena en una sola pasada
- Funciona con poquísimos datos
- Naturalmente probabilístico

Limitaciones

- Supuesto de independencia raramente cierto
- Probabilidades mal calibradas
- Sensible a features irrelevantes

Naive Bayes — formulación

La matemática

$$P(C | \mathbf{x}) = P(C) \cdot P(\mathbf{x} | C) / P(\mathbf{x})$$

Teorema de Bayes: posterior es proporcional a prior por verosimilitud.

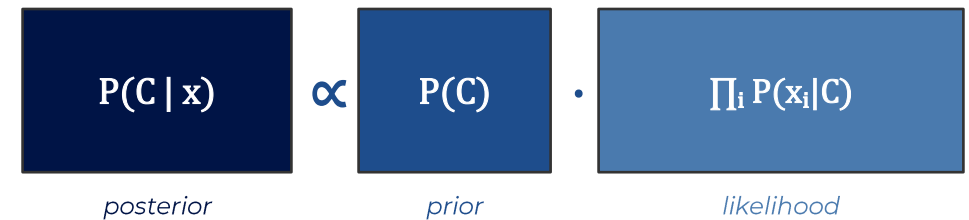
$$P(\mathbf{x} | C) \approx \prod_i P(x_i | C)$$

Supuesto naive: la verosimilitud conjunta se factoriza en el producto sobre features.

$$\hat{c} = \operatorname{argmax}_k P(C_k) \cdot \prod_i P(x_i | C_k)$$

Regla de decisión MAP: elegir la clase con mayor probabilidad posterior.

Lectura visual



Regla de decisión

$$\hat{c} = \operatorname{argmax}_k P(C_k) \cdot \prod_i P(x_i | C_k)$$

Cómo se entrena

El entrenamiento consiste en estimar $P(C)$ (frecuencias de clase en el train) y $P(x_i | C)$ (parámetros de la distribución supuesta por feature: media y varianza si es Gaussiana; frecuencias si es multinomial). No hay optimización iterativa: las estimaciones son directas a partir de las frecuencias.

Naive Bayes paso a paso

Clasificamos un correo nuevo como SPAM o HAM con un dataset de 12 mensajes etiquetados. Vocabulario: 'gratis', 'oferta', 'reunión', 'informe'.

1. Los datos (n = 12)

#	Clase	gratis	oferta	reunión	informe
1	SPAM	1	1	0	0
2	SPAM	1	1	0	0
3	SPAM	1	1	0	1
4	SPAM	1	1	1	0
5	SPAM	1	0	0	0
6	SPAM	0	0	0	0
7	HAM	0	0	1	1
8	HAM	0	0	1	1
9	HAM	0	1	1	1
10	HAM	0	1	1	0
11	HAM	0	0	1	1
12	HAM	1	0	0	0

Correo a clasificar:

$\mathbf{x} = \{\text{gratis}=1, \text{oferta}=1, \text{reunión}=0, \text{informe}=0\}$

2. Los cálculos

Priors

$$P(\text{SPAM}) = 6/12 = 0,50 \quad \cdot \quad P(\text{HAM}) = 6/12 = 0,50$$

Likelihoods (frecuencias directas)

$$P(\text{gratis}=1 \mid \text{SPAM}) = 5/6 \approx 0,833$$

$$P(\text{oferta}=1 \mid \text{SPAM}) = 4/6 \approx 0,667$$

$$P(\text{reunión}=0 \mid \text{SPAM}) = 5/6 \approx 0,833$$

$$P(\text{informe}=0 \mid \text{SPAM}) = 5/6 \approx 0,833$$

$$P(\text{gratis}=1 \mid \text{HAM}) = 1/6 \approx 0,167$$

$$P(\text{oferta}=1 \mid \text{HAM}) = 2/6 \approx 0,333$$

$$P(\text{reunión}=0 \mid \text{HAM}) = 1/6 \approx 0,167$$

$$P(\text{informe}=0 \mid \text{HAM}) = 2/6 \approx 0,333$$

Posteriors (proporcionales)

$$P(\text{SPAM} \mid \mathbf{x}) \propto 0,50 \cdot 0,833 \cdot 0,667 \cdot 0,833 \cdot 0,833 \approx 0,193$$

$$P(\text{HAM} \mid \mathbf{x}) \propto 0,50 \cdot 0,167 \cdot 0,333 \cdot 0,167 \cdot 0,333 \approx 0,00154$$

3. La decisión

SPAM

(según $\hat{c} = \text{argmax}_k \text{posterior}$)

0,193 / 0,00154

≈ 125×

más probable SPAM que HAM

Por qué funciona

Ninguna probabilidad es cero porque cada palabra aparece al menos una vez en cada clase. El producto se calcula limpio.

¿Y si una palabra nunca aparece en una clase?

Cuando un conteo es cero, el producto de likelihoods se anula y la clase queda descartada por una sola palabra. La corrección de Laplace soluciona ese colapso.

EL PROBLEMA

Conteos cero

Supongamos que en el train no hubo ningún correo HAM con la palabra 'gratis'. Entonces:

$$P(\text{gratis}=1 \mid \text{HAM}) = 0 / 6 = 0$$

Y al multiplicar dentro del posterior:

$$P(\text{HAM} \mid x) \propto P(\text{HAM}) \cdot 0 \cdot P(\text{oferta} \mid \text{HAM}) \cdot \dots = 0$$

Una sola palabra desconocida anula toda la clase HAM, sin importar cuán probables sean las demás features. La ausencia de evidencia se convierte erróneamente en evidencia de imposibilidad.

LA SOLUCIÓN

Suavizado de Laplace

Sumamos un conteo ficticio α (típicamente 1) a cada combinación posible, antes de calcular la frecuencia:

$$P(x_i=v \mid C) = (\text{conteo} + \alpha) / (\text{total} + \alpha \cdot K)$$

α : conteo ficticio (1 = Laplace) · K : valores posibles de la feature

Volviendo al caso del cero:

$$P(\text{gratis}=1 \mid \text{HAM}) = (0+1) / (6+2) = 1/8 = 0,125$$

Distinto de cero: el producto ya no se anula, y el clasificador puede comparar posteriors normalmente.

Naive Bayes — caso de uso

Filtrar spam en un flujo de correos en tiempo real para un proveedor de email corporativo.

El contexto

Necesitamos clasificar millones de mensajes por día con latencia mínima, reentrenando seguido porque el spam cambia de patrón rápidamente.

Los datos

Bag-of-words sobre el cuerpo y asunto de cada mensaje, con vocabularios de decenas de miles de términos. Etiqueta binaria: spam o ham.

Por qué este modelo

Multinomial Naive Bayes es históricamente la baseline para clasificación de texto: maneja vocabularios enormes, entrena en una pasada y escala a producción sin esfuerzo.

Cómo se evalúa

Precision en la clase spam (no marcar ham legítimo como spam es caro), recall, y F1. Monitorear deriva de la distribución de tokens en el tiempo.

A TENER EN CUENTA *El supuesto de independencia entre palabras es claramente falso, pero el ranking de probabilidades suele ser correcto aunque las probabilidades en sí estén mal calibradas. No interpretar los valores como probabilidades reales.*

Naive Bayes — en scikit-learn

Variante multinomial sobre texto vectorizado con TF-IDF. Para features continuas se usaría GaussianNB.

```
python

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

# Vectorización TF-IDF + clasificador multinomial
modelo = Pipeline([
    ('tfidf', TfidfVectorizer(min_df=2, ngram_range=(1,2))),
    ('nb', MultinomialNB(alpha=1.0)),
])

# Ajuste sobre texto crudo
modelo.fit(X_train_text, y_train)

# Predicción y reporte por clase
y_pred = modelo.predict(X_test_text)
print(classification_report(y_test, y_pred))
```

Hiperparámetros clave

alpha

Suavizado de Laplace; $\alpha=1.0$ evita probabilidades nulas en palabras no vistas.

fit_prior

Si False, asume priors uniformes. Útil con clases muy desbalanceadas.

class_prior

Permite fijar a mano las probabilidades a priori de cada clase.

Variante

MultinomialNB (texto), GaussianNB (continuas), BernoulliNB (binarias).

ENSEMBLE SECUENCIAL

Gradient Boosting

06

05

ALGORITMO · INTUICIÓN

Gradient Boosting

Ensemble de árboles débiles entrenados secuencialmente, donde cada uno corrige los errores del anterior.

La idea

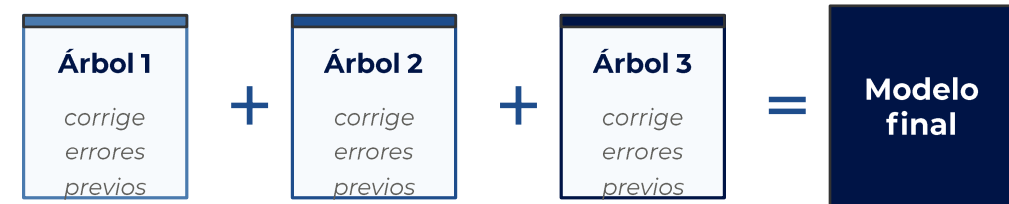
A diferencia de Random Forest (que entrena árboles en paralelo y los promedia), boosting los entrena en serie. Cada árbol nuevo se ajusta a los residuos del modelo acumulado hasta el momento — es decir, aprende exactamente lo que el ensemble actual está errando. La predicción final es la suma ponderada de todos los árboles. Por eso suele dominar competencias sobre datos tabulares: XGBoost, LightGBM y CatBoost son sus implementaciones modernas.

Supuestos clave

- Aprendices débiles (típicamente árboles de profundidad 3-8)
- Función de pérdida diferenciable
- Datos representativos: extrapola mal fuera del rango visto

Esquema

Aprendizaje secuencial: cada modelo corrige los residuos del anterior



Bagging (RF): árboles en paralelo · **Boosting:** árboles en serie

Fortalezas

- Suele dominar en datos tabulares
- Maneja interacciones y no linealidad
- Acepta mezcla de variables sin escalado

Limitaciones

- Riesgo de overfitting sin early stopping
- Más lento de entrenar que RF
- Hiperparámetros sensibles

Gradient Boosting — formulación

La matemática

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + v \cdot h_m(\mathbf{x})$$

El modelo en la iteración m es el del paso anterior más un nuevo árbol h , escalado por la tasa de aprendizaje v .

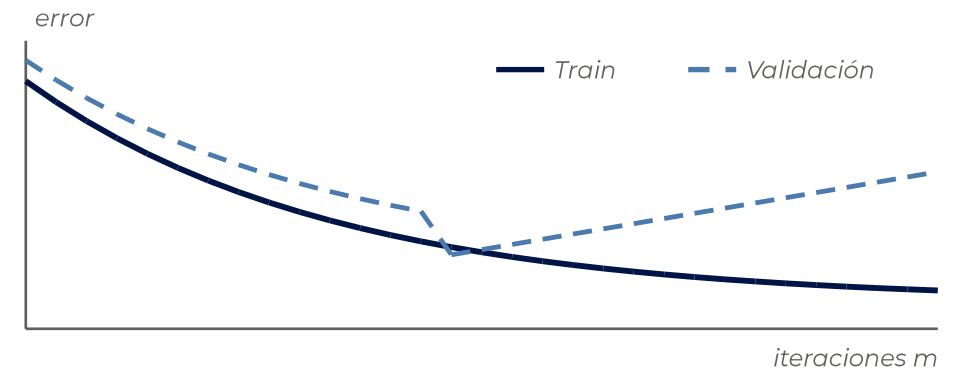
$$h_m \approx -\partial L(\mathbf{y}, F(\mathbf{x})) / \partial F(\mathbf{x}) |_{\{F=F_{m-1}\}}$$

Cada árbol se ajusta al gradiente negativo de la pérdida: por eso 'gradient' boosting.

$$v \in (0, 1] \cdot M \uparrow \cdot \text{profundidad} \downarrow$$

Triángulo de regularización: aprendizaje chico, más iteraciones, árboles poco profundos.

Lectura visual



$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + v \cdot h_m(\mathbf{x})$$

Cómo se entrena

Cada iteración ajusta un árbol a la dirección de descenso más rápido de la pérdida; el árbol se agrega al ensemble con un peso v pequeño (shrinkage). El control de overfitting viene de tres palancas combinadas: tasa de aprendizaje baja, número de iteraciones controlado por early stopping en validación, y árboles individualmente débiles.

Gradient Boosting — caso de uso

Predecir la probabilidad de churn de clientes en una telco con 36 meses de historia transaccional.

El contexto

El equipo de retención necesita un score mensual de clientes que se irán en los próximos 60 días, para priorizar campañas de fidelización por valor.

Los datos

≈ 2 millones de clientes con 150+ features: uso, planes, facturación, llamadas a soporte, comportamiento web, demográficas. Mezcla densa-rala de tipos.

Por qué este modelo

Datos tabulares heterogéneos, muchas interacciones plausibles (uso × plan × antigüedad), y exigencia de máxima discriminación: el caso típico donde Gradient Boosting domina a todo lo demás.

Cómo se evalúa

AUC y lift en el top-10% de scores (donde se concentra la campaña). Monitorear estabilidad de la distribución de scores entre meses.

A TENER EN CUENTA *Es una caja gris. Para auditar decisiones y diseñar la conversación con el cliente, combinar con SHAP para explicar cada predicción individual; el solo AUC alto no convence al área de negocio.*

Gradient Boosting — en scikit-learn

HistGradientBoostingClassifier de sklearn: rápido, maneja NaN, comparable a XGBoost/LightGBM. Con early stopping en validación.

```
python

from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score

# Modelo con early stopping en val interna
modelo = HistGradientBoostingClassifier(
    learning_rate=0.05,
    max_iter=1000,
    max_depth=6,
    early_stopping=True, validation_fraction=0.15,
    n_iter_no_change=20, random_state=42,
)

modelo.fit(X_train, y_train)

y_proba = modelo.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, y_proba)
```

Hiperparámetros clave

learning_rate

∇: peso de cada árbol nuevo. Chico (0.01–0.1) + más iteraciones suele ganar.

max_iter

Cantidad máxima de árboles. Acotar overfitting con early_stopping.

max_depth

Profundidad de cada árbol. 3–8 típicamente; más profundo = más interacciones.

early_stopping

Corta el entrenamiento cuando la pérdida en validación deja de mejorar.

VISTAS DE CONJUNTO

Cuándo conviene cada uno

06

Cuándo conviene cada uno

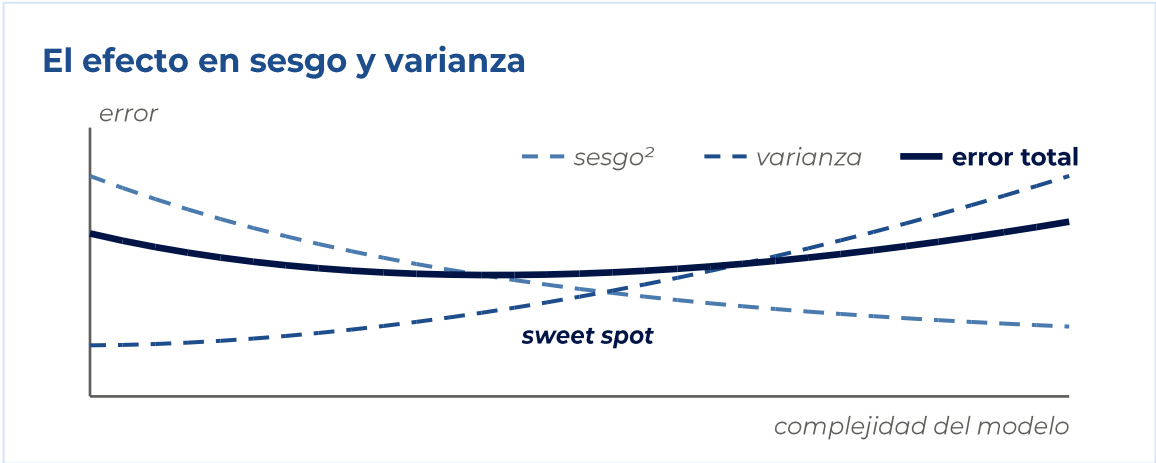
Los seis clasificadores que cubrimos en el curso, comparados según familia, supuestos, interpretabilidad y cuándo conviene elegirlos.

	Familia	Idea central	Interpretabilidad	Cuándo conviene
Reg. Logística	Lineal / probabilístico	Frontera lineal vía sigmoide	Alta: coeficientes leíbles	Baseline; señales aproximadamente lineales en log-odds
Naive Bayes	Probabilístico generativo	Bayes con features independientes	Media: probabilidades por clase	Texto/spam, alta dimensión, pocos datos
Gradient Boosting	Ensemble secuencial	Suma de árboles que corrigen residuos	Baja: caja gris (con SHAP mejora)	Datos tabulares mixtos; máxima performance
Árbol de Decisión	Particionamiento	Splits sucesivos por ganancia	Muy alta: reglas explícitas	Interpretabilidad prioritaria, pocas features
KNN	Basado en instancias	Voto de los k vecinos cercanos	Media: depende del ejemplo	Fronteras irregulares; datasets chicos
Random Forest	Ensemble por bagging	Promedio de árboles aleatorizados	Baja: importancia de features	Robustez out-of-the-box, ruido moderado

Hiperparámetros: qué son y cómo se eligen

Decisiones de configuración que el modelo NO aprende de los datos: las fijamos antes de entrenar. Controlan la complejidad y, por ende, el equilibrio entre sesgo y varianza.

Parámetro	Hiperparámetro
Lo aprende el modelo durante el entrenamiento	Lo fijamos nosotros antes de entrenar
Cambia al optimizar la pérdida	Cambia solo si decidimos probar otro valor
Ej.: coeficientes β de la logística	Ej.: C, max_depth, learning_rate



¿Cómo se eligen?

Grid search Probar todas las combinaciones de una grilla predefinida.	Random search Sortear N combinaciones al azar. Más eficiente con muchos hiperparámetros.	Validación cruzada Evaluar cada combinación con k-fold CV en vez de un solo split.	Bayesian search Modelar la performance y elegir el próximo punto con más información esperada.
---	--	--	--

REGLA PRÁCTICA Nunca evaluar la elección de hiperparámetros en el test set: usar siempre validación cruzada o un holdout dedicado.

CIERRE

¡Gracias!

Próxima clase: regularización, calibración de probabilidades y elección de umbral.

Profesor Ing. Pablo F. Sciolla

Universidad de San Andrés — Analítica de Datos

0
7